

被破坏,但数据可能不正确,这是由于事务的运行被非正常终止造成的。

恢复的基本原理十分简单。可以用一个词来概括——冗余。也就是说,数据库中任何一部分被破坏或不正确的数据可以根据存储在系统别处的冗余数据来重建。尽管恢复的基本原理很简单,但实现技术的细节却相当复杂。下面略去一些细节,介绍数据库恢复的实现技术。

11.4 恢复的实现技术

恢复机制涉及两个关键问题,即如何建立冗余数据,以及如何利用这些冗余数据实施数据库恢复。

建立冗余数据最常用的技术是数据转储(dump)和登记日志文件。通常在一个数据库系统中这两种方法是一起使用的。

11.4.1 数据转储

数据转储是数据库恢复中采用的基本技术。所谓数据转储,即数据库管理员定期将整个数据库复制到磁带、磁盘或其他存储介质上保存的过程。这些备用的数据称为后备副本或后援副本。

当数据库遭到破坏后可以将后备副本重新装入,但重装后备副本只能将数据库恢复到转储时的状态,要想恢复到故障发生时的状态,必须重新运行自转储以后的所有更新事务。例如,在图 11.1 中,系统在 T_a 时刻停止运行事务,进行数据库转储,在 T_b 时刻转储完毕,得到 T_b 时刻的数据库一致性副本。系统运行到 T_f 时刻发生故障。为恢复数据库,首先由数据库管理员重装数据库后备副本,重装的后备副本是 T_a 时刻转储的副本,将数据库恢复至 T_b 时刻的状态,然后重新运行自 $T_b \sim T_f$ 时刻所有已提交的更新事务,这样就把数据库恢复到故障发生前的一致状态了。

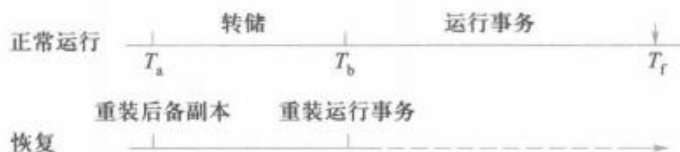


图 11.1 转储和恢复示例

数据转储是十分耗费时间和资源的,不能频繁进行。数据库管理员应根据数据库使用情况确定一个适当的转储周期。

数据转储按数据库的状态可分为静态转储和动态转储。

静态转储是在系统中无运行事务时进行的转储操作。静态转储操作开始的时刻数据库处于一致性状态,在转储期间不允许(或不存在)对数据库的任何存取、修改活动。显然,静态转储得到的一定是一个数据一致性的副本。

静态转储简单,但转储必须等待正运行的用户事务结束才能进行。同样,新的事务必须等待转储结束才能执行。显然,这会降低数据库的可用性。

动态转储是指转储期间允许对数据库进行存取或修改,即转储和用户事务可以并发执行。

动态转储可以克服静态转储的缺点,它不用等待正在运行的用户事务结束,也不会影响新事务的运行。但是,转储结束后后备副本上的数据并不能保证正确有效。例如,在转储期间的某个时刻 T_c ,系统把数据 $A = 100$ 转储到磁带上,而在下一时刻 T_d ,某一事务将 A 改为 200。转储结束后,后备副本上的 A 已是过时的数据了。

为此,必须把数据转储期间各事务对数据库的修改活动登记下来,建立日志文件(log file)。这样,后备副本加上日志文件就能把数据库恢复到某一时刻的正确状态。

数据转储还可以分为海量转储和增量转储两种方式。**海量转储**是指每次转储全部数据库,增量转储则指每次只转储上一次转储后更新过的数据。从恢复角度看,使用海量转储得到的后备副本进行恢复一般会更方便些。但如果数据库很大,事务处理又十分频繁,则增量转储方式更实用且更有效。

数据转储有两种方式,分别可以在两种状态下进行,因此数据转储方法可以分为4类:动态海量转储、动态增量转储、静态海量转储和静态增量转储,如表 11.1 所示。

表 11.1 数据转储方法

转储方式	转储状态	
	动态转储	静态转储
海量转储	动态海量转储	静态海量转储
增量转储	动态增量转储	静态增量转储

11.4.2 登记日志文件

日志文件是用来记录事务对数据库的更新操作的文件。不同数据库系统采用的日志文件格式并不完全一样。概括起来日志文件主要有两种格式:以记录为单位的日志文件和以数据块为单位的日志文件。

1. 日志文件的格式和内容

对于以记录为单位的日志文件,日志文件中需要登记的内容包括:

- ① 各个事务的开始标记(BEGIN TRANSACTION)。
- ② 各个事务的结束标记(COMMIT 或 ROLLBACK)。
- ③ 各个事务的所有更新操作。

这里每个事务的开始标记、结束标记和每个更新操作均作为日志文件中的一个日志记录。

每个日志记录的内容主要包括:

- ① 事务标识(标明是哪个事务)。

- ② 操作的类型(插入、删除或修改)。
- ③ 操作对象(记录内部标识)。
- ④ 更新前数据的旧值(对插入操作而言,此项为空值)。
- ⑤ 更新后数据的新值(对删除操作而言,此项为空值)。

对于以数据块为单位的日志文件,日志记录的内容包括事务标识和被更新的数据块。由于将更新前的整个块和更新后的整个块都放入日志文件中,操作类型和操作对象等信息就不必放入日志记录中了。

2. 日志文件的作用

日志文件在数据库恢复中起着非常重要的作用,可以用来进行事务故障恢复和系统故障恢复,并协助后备副本进行介质故障恢复。

- ① 事务故障恢复和系统故障恢复必须用日志文件。
- ② 在动态转储方式中必须建立日志文件,后备副本和日志文件结合起来才能有效地恢复数据库。
- ③ 在静态转储方式中也可以建立日志文件,当数据库毁坏后可重新装入后备副本,把数据库恢复到转储结束时刻的正确状态,然后利用日志文件把已完成的事务进行重做处理,对故障发生时未完成的事务进行撤销处理。这样,不必重新运行那些已完成的事务程序就可把数据库恢复到故障前某一时刻的正确状态,如图 11.2 所示。

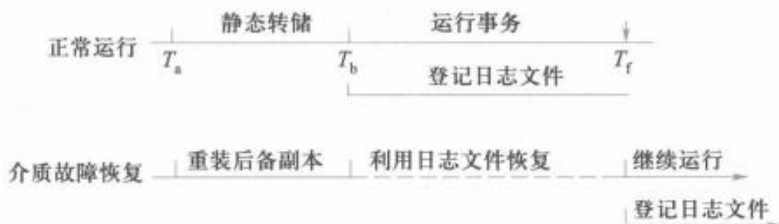


图 11.2 利用日志文件恢复

3. 登记日志文件

为保证数据库是可恢复的,登记日志文件时必须遵循两条原则:

- ① 登记的次序严格按并发事务执行的时间次序。
- ② 必须先写日志文件,后写数据库。

把对数据的修改写到数据库中和把表示这个修改的日志记录写到日志文件中是两个不同的操作。有可能在这两个操作之间发生故障,即这两个写操作只完成了一个。如果先写了数据库修改,而在日志文件中没有登记这个修改,则以后就无法恢复这个修改了。如果先写日志,但没有修改数据库,按日志文件恢复时只不过是多执行一次不必要的 UNDO 操作,并不会影响数据库的正确性。所以为了安全,一定要先写日志文件,即首先把日志记录写到日志文件中,然后再写数据库的修改。这就是“先写日志文件”的原则。

11.5 恢复策略

若系统运行过程中发生故障,利用数据库后备副本和日志文件就可以将数据库恢复到故障前的某个一致性状态。不同故障,其恢复策略和方法各不相同。

1. 事务故障的恢复

事务故障是指事务在运行至正常终止点前被终止,这时恢复子系统应利用日志文件撤销(UNDO)此事务已对数据库进行的修改。事务故障的恢复是由系统自动完成的,对用户隐蔽。系统的恢复步骤如下:

① 反向扫描日志文件(即从最后向前扫描日志文件),查找该事务的更新操作。

② 对该事务的更新操作执行逆操作,将日志记录中“更新前的值”写入数据库。这样,如果记录中是插入操作,则相当于做删除操作(因此时“更新前的值”为空);若记录中是删除操作,则做插入操作;若是修改操作,则相当于用修改前值代替修改后值。

③ 继续反向扫描日志文件,查找该事务的其他更新操作并做同样处理。

④ 如此处理下去,直至读到此事务的开始标记,事务故障恢复即完成。

2. 系统故障的恢复

前面已讲过,系统故障造成数据库不一致的原因有两个:一是未完成事务对数据库的更新可能已写入数据库,二是已提交事务对数据库的更新可能还留在缓冲区来不及写入数据库。因此恢复操作就是要撤销故障发生时未完成的事务,重做已完成的事务。

系统故障的恢复是由系统在重新启动时自动完成的,不需要用户干预。

系统的恢复步骤如下:

① 正向扫描日志文件(即从头扫描日志文件),找出在故障发生前已经提交的事务(这些事务既有 BEGIN TRANSACTION 记录,也有 COMMIT 记录),将其事务标识记入重做队列(REDO-LIST);同时找出故障发生时尚未完成的事务(这些事务只有 BEGIN TRANSACTION 记录,无相应的 COMMIT 记录),将其事务标识记入撤销队列(UNDO-LIST)。

② 对撤销队列中的各个事务进行撤销(UNDO)处理。进行撤销处理的方法是:反向扫描日志文件,对每个撤销事务的更新操作执行逆操作,即将日志记录中“更新前的值”写入数据库。

③ 对重做队列中的各个事务进行重做处理。进行重做处理的方法是:正向扫描日志文件,对每个重做事务重新执行日志文件登记的操作,即将日志记录中“更新后的值”写入数据库。

3. 介质故障的恢复

发生介质故障后,磁盘上的物理数据和日志文件被破坏,这是最严重的一种故障,其恢复方法是重装数据库,然后重做已完成的事务。

① 装入最新的数据库后备副本(离故障发生时刻最近的转储副本),使数据库恢复到最近一次转储时的一致性状态。对于动态转储的数据库副本,还需同时装入转储开始时刻的日志文